

WEEK 1 LAB

Create Your Architectural Trail

Lab 01: Team repository, charter, issue board, and first discovery issue

MODE	IN CLASS	TOOLS	SEMESTER
Individual	75 minutes	GitHub + Desktop + VS Code	One lasting repo

THE IDEA

Your architectural trail is the connected record of why the team acted, who owned the work, what evidence supported it, what changed, and who reviewed the change. The repository is not merely a place to store code.

Why individual repositories are the right Week 1 choice

Modern software architecture is collaborative, but foundational fluency should not belong to one designated repository owner. In this course, collaboration means that everyone can author a change and everyone can review a change. Week 2 will be the principal team workshop; Week 1 establishes equal technical practice and equal grading evidence.

Course week	How collaboration works	What every student does
Week 1	Reciprocal peer help and pull-request review.	Creates and operates an individual semester repository.
Week 2	Teams perform Design Thinking wall work and agree on discovery evidence.	Participates in the same hopes/fears, clustering, empathy, and synthesis activities.
Week 3 and later	Students may compare ideas and reuse approved team evidence.	Performs the same graded GitHub and technical lab actions in the individual repository.

The four places you will work

Tool	What it is	What you do there
GitHub website	The online or remote home of the repository.	Create the repository, issue, pull request, review, and Project board.
GitHub Desktop	A visual application that synchronizes local and online Git work.	Clone, see changed files, commit, publish a branch, pull, and push.
Visual Studio Code	The editor on your computer.	Open the cloned folder and create or edit Markdown files.

Tool	What it is	What you do there
Finder / File Explorer	Your computer's ordinary folder viewer.	Confirm where the local repository folder is stored; do not move it during the lab.

TWO COPIES, ONE PROJECT

GitHub holds the remote copy online. GitHub Desktop clones a local copy onto your computer. VS Code edits the local files. A commit records a checkpoint; Push origin sends that checkpoint to GitHub.

Timing and finish line

Part	Minutes	Finish line
Pre-lab readiness	Before class	Accounts and applications open
1. Create and clone	15	Private repository exists online and locally
2. Open issue and branch	10	Issue #1 is connected to a working branch
3. Build repository front door	20	README, charter, AI log, src, and tests files
4. Commit, push, and pull request	10	One linked pull request
5. Reciprocal peer review	10	One review given and one received
6. Semester Project board	8	Four Week 1–4 issues on a board
7. Verify and submit	2	Repository URL and evidence ready

What you will submit

1. Your individual private repository URL; grant the instructor access if requested.
2. Issue #1 and the merged Week 1 pull request.
3. The peer pull request on which you submitted a useful review.
4. Your linked CampusConnect Semester Board containing the Week 1–4 issues.
5. One screenshot showing the repository file list after the Week 1 pull request is merged.

Pre-lab readiness — complete before the 75-minute clock

IF AN APPLICATION IS ALREADY INSTALLED

Open it and confirm that you can sign in. Do not uninstall or reinstall a working application merely to follow these directions.

A. Create or verify your GitHub account

Use this tool	Where you should be	Visible finish line
Web browser	GitHub sign-in page	You can see your avatar and username in the upper-right corner.

1. Open GitHub: <https://github.com/> and select Sign up or Sign in.
2. Use a professional username that you are comfortable submitting to the instructor.
3. Verify the email address GitHub sends you. Some repository features remain limited until verification is complete.
4. Write your exact GitHub username here: _____.
5. If you need account help, open: <https://docs.github.com/en/account-and-profile/how-to/account-management/creating-an-account-on-github>.

B. Install and sign in to GitHub Desktop

Use this tool	Where you should be	Visible finish line
Web browser, then GitHub Desktop	The GitHub Desktop download page	GitHub Desktop opens and shows your GitHub.com account.

1. Open the official download page: <https://desktop.github.com/>.
2. On macOS, select Download for macOS, open the ZIP from Downloads, and move or open GitHub Desktop. On Windows, select Download for Windows and run the installer.
3. Open GitHub Desktop. On a Mac, use GitHub Desktop > Settings > Accounts. On Windows, use File > Options > Accounts.
4. Select Sign in to GitHub.com, then Continue with browser. Approve the request in the browser and return to GitHub Desktop.
5. Official installation help: <https://docs.github.com/en/desktop/installing-and-authenticating-to-github-desktop/installing-github-desktop>.

6. Official sign-in help: <https://docs.github.com/en/desktop/installing-and-authenticating-to-github-desktop/authenticating-to-github-in-github-desktop>.

C. Install or verify Visual Studio Code

Use this tool	Where you should be	Visible finish line
Web browser, then VS Code	The official Visual Studio Code download page	VS Code opens to its Welcome screen.

1. Open: <https://code.visualstudio.com/download>. Select the correct installer for your computer.
2. macOS instructions: <https://code.visualstudio.com/docs/setup/mac>.
3. Windows instructions: <https://code.visualstudio.com/docs/setup/windows>.
4. Open VS Code once. No extension, Copilot subscription, Python environment, or Terminal command is required for this lab.

ASK FOR HELP BEFORE CLASS STARTS IF

You cannot sign in, GitHub Desktop shows the wrong account, a school-managed computer blocks installation, or VS Code cannot open. The browser-only fallback at the end of this guide keeps you moving, but the applications are the preferred semester setup.

Part 1 — Create and clone your repository (15 minutes)

Step 1 — Create the private remote repository

Use this tool	Where you should be	Visible finish line
GitHub website	New repository page	A private repository with README.md and a Python .gitignore.

1. Open the direct creation page: <https://github.com/new>.
2. Under Owner, select your own GitHub account—not a classmate’s account.
3. In Repository name, type comp7431-<your-github-username>-campusconnect. Replace the angle-bracket text. Example: comp7431-apisch-campusconnect.
4. In Description, type: Individual COMP-7431 architectural trail for CampusConnect.
5. Select Private. A private repository is not a secure vault, but it avoids publishing unfinished class work.
6. Turn on Add a README file.
7. Under Add .gitignore, select Python. This creates starter exclusions for Python artifacts even though substantial coding comes later.
8. Leave Choose a license set to None. A public software license is a permission decision, not decoration.
9. Select Create repository.
10. Official GitHub directions: <https://docs.github.com/en/repositories/creating-and-managing-repositories/creating-a-new-repository>.

CHECKPOINT

Your browser should now show the Code tab with README.md and .gitignore. Copy the repository URL from the browser address bar and paste it into your assignment notes.

Step 2 — Clone the repository onto your computer

Use this tool	Where you should be	Visible finish line
GitHub website and GitHub Desktop	Your repository’s Code tab	GitHub Desktop shows Current repository = your CampusConnect repository.

1. On the repository Code tab, select the green Code button above the file list.

2. Select Open with GitHub Desktop. If your browser asks permission to open GitHub Desktop, allow it.
3. In GitHub Desktop, inspect Local path. Use an easy-to-find location such as Documents/GitHub. Do not place the repository inside iCloud Drive, Google Drive, Dropbox, or OneDrive for this lab; two synchronization systems can compete.
4. Select Clone.
5. When cloning finishes, select Repository > Show in Finder on macOS or Repository > Show in Explorer on Windows.
6. Look at the local folder, then close Finder or File Explorer without renaming or moving anything.
7. Official cloning help: <https://docs.github.com/en/desktop/adding-and-cloning-repositories/cloning-a-repository-from-github-to-github-desktop>.

WHAT CLONE MEANS

Clone makes a working copy on your computer and remembers which GitHub repository it came from. You do not need to download ZIP files for normal semester work.

Part 2 — Open Issue #1 and create its branch (10 minutes)

Step 3 — Create the Week 1 setup issue

Use this tool	Where you should be	Visible finish line
GitHub website	Your repository > Issues	Open issue titled Week 1: establish my architectural trail.

1. Return to the GitHub website and open your repository.
2. Select the Issues tab beneath the repository name. If GitHub shows a welcome page, select Set up issues.
3. Select New issue. If templates appear, select Open a blank issue.
4. In Title, type: Week 1: establish my architectural trail
5. Paste the issue starter below into the description.
6. In the right sidebar, select Assignees, then assign yourself.
7. Select Submit new issue.
8. Official issue directions: <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-an-issue>.

COPY / PASTE — Issue #1 description

Purpose

Create my individual semester repository and practice the complete GitHub trail.

Done when

- [] Repository contains the README, Project Charter, and Engineering/AI Use Log.
- [] Repository contains src/README.md and tests/README.md placeholders.
- [] Work is committed on branch docs/week-1-setup.
- [] A pull request links to this issue.
- [] An assigned peer leaves one useful review comment.
- [] I respond to the comment and merge the pull request.
- [] My semester Project board contains the Week 1-4 issues.

Safety

No private student information, credentials, secrets, API keys, or unapproved campus documents will be stored in this repository.

Step 4 — Create a branch directly from Issue #1

Use this tool	Where you should be	Visible finish line
GitHub website	The open Week 1 issue	The issue sidebar shows linked branch docs/week-1-setup.

1. Keep Issue #1 open.
2. In the right sidebar, find Development. Select Create a branch.
3. In Branch name, type exactly: docs/week-1-setup
4. Under What's next, choose Checkout locally if GitHub offers it. If the choice is absent, create the branch anyway.
5. Select Create branch.
6. Open GitHub Desktop. Select Fetch origin. Then select Current Branch and choose docs/week-1-setup if it is not already selected.
7. Look at the top of GitHub Desktop. Do not continue until Current Branch says docs/week-1-setup rather than main.
8. Official branch-from-issue directions: <https://docs.github.com/en/issues/tracking-your-work-with-issues/using-issues/creating-a-branch-for-an-issue>.

WHY THE BRANCH MATTERS

main is the stable semester record. The branch is a safe workspace for one coherent change. Linking the branch to Issue #1 explains why the work exists.

Part 3 — Build the repository front door (20 minutes)

Step 5 — Open the cloned folder in VS Code

Use this tool	Where you should be	Visible finish line
GitHub Desktop and Visual Studio Code	GitHub Desktop on branch docs/week-1-setup	VS Code Explorer shows README.md and .gitignore.

1. In GitHub Desktop, select Repository > Open in Visual Studio Code.
2. If that exact menu item is unavailable, select Repository > Show in Finder/Explorer. Open VS Code, select File > Open Folder, choose the CampusConnect folder, then select Open.
3. In VS Code, use the Explorer panel on the left. Confirm that README.md and .gitignore appear.
4. If VS Code asks whether you trust the authors, choose trust only because this is the repository you just created. Do not automatically trust unfamiliar downloaded repositories.

Step 6 — Replace README.md

Use this tool	Where you should be	Visible finish line
Visual Studio Code	README.md in the Explorer	README.md describes purpose, boundary, folder map, and work rule.

1. In the VS Code Explorer, select README.md.
2. Select all existing text inside the editor and replace it with the starter below.
3. Replace no content except where the starter tells you to personalize it.
4. Save: Command+S on macOS or Ctrl+S on Windows.

COPY / PASTE — README.md

```
# CampusConnect

## Purpose
This repository is my semester-long engineering trail for COMP-7431. It will show
how discovery evidence becomes requirements, architecture decisions, code, tests,
and revisions.

## Current problem hypothesis
Students may have difficulty finding timely, trustworthy IT Support information.
This is a hypothesis until Week 2 evidence supports, revises, or rejects it.

## Current boundary
IN: learning artifacts, anonymized class evidence, requirements, architecture notes,
small prototypes, tests, and reflection.
OUT: real student records, passwords, API keys, private email, production systems,
and unapproved campus documents.

## Repository map
- docs/PROJECT_CHARTER.md - purpose, working rules, and safety boundaries
- docs/ENGINEERING_AI_USE_LOG.md - transparent record of AI assistance
- docs/requirements/ - requirements beginning in Week 3
- docs/architecture/ - diagrams and decisions beginning in Week 4
- src/ - prototype code later in the semester
- tests/ - tests later in the semester

## Working rule
Work begins with an issue. Changes are made on a branch, reviewed in a pull request,
and merged only after the evidence is understandable.
```

Step 7 — Create the docs folder and two Markdown files

Use this tool	Where you should be	Visible finish line
Visual Studio Code	Explorer panel inside the repository	docs contains PROJECT_CHARTER.md and ENGINEERING_AI_USE_LOG.md.

1. In the Explorer, select the New Folder icon. Name the folder docs.
2. Select the docs folder once so it is highlighted.
3. Select the New File icon. Name the file PROJECT_CHARTER.md.
4. Paste the Project Charter starter below, replace <YOUR NAME>, and save.

5. Select the docs folder again, select New File, and name it ENGINEERING_AI_USE_LOG.md.
6. Paste the AI Use Log starter below. Replace the date and save.

COPY / PASTE — docs/PROJECT_CHARTER.md

```
# Individual Project Charter

## Student
<YOUR NAME>

## Repository purpose
Preserve a reviewable engineering trail for the CampusConnect semester project.

## Working rules
1. Begin substantial work with a GitHub issue.
2. Create a short, descriptive branch from the issue.
3. Commit a coherent change with a meaningful message.
4. Open a pull request and connect it to the issue.
5. Request a reciprocal peer review when the lab requires one.
6. Never commit directly to main after Week 1 setup.

## Evidence rule
Separate evidence from inference and assumption. Link the source when one is approved.

## AI rule
Log the tool, context shared, output retained, human changes, and verification.

## Data and secrets rule
Never commit names from interviews, grades, accommodations, advising notes, private email, passwords, tokens, API keys, production data, or unapproved IT documents.
```

COPY / PASTE — docs/ENGINEERING_AI_USE_LOG.md

```
# Engineering / AI Use Log

| Date | Tool | Task | Context shared | Output retained | Human verification | Link |
| ---|---|---|---|---|---|---|
| YYYY-MM-DD | None | Week 1 repository setup | None | None | I created and checked the repository trail myself. | #1 |
```

Step 8 — Create src and tests placeholders

Use this tool	Where you should be	Visible finish line
Visual Studio Code	Explorer panel inside the repository	src/README.md and tests/README.md exist; no code is required.

1. Create a folder named src. Inside it, create README.md.
2. In src/README.md, type: Prototype source code will be added later in the semester. Save.
3. Create a folder named tests. Inside it, create README.md.
4. In tests/README.md, type: Test cases and automated tests will be added later in the semester. Save.
5. Do not create empty folders only. Git does not normally preserve an empty folder; the small README keeps each folder visible.

Step 9 — Strengthen .gitignore

Use this tool	Where you should be	Visible finish line
Visual Studio Code	.gitignore in the Explorer	Common secret and local-environment files are excluded.

1. Select .gitignore.
2. Scroll to the bottom and add the lines below.
3. Save the file.
4. A .gitignore reduces accidental commits; it does not erase a secret that was already committed. Never paste a real secret into the repository.

COPY / PASTE — Minimum .gitignore additions

```
.env
.env.*
!.env.example
*.key
*.pem
secrets/
credentials.*
.DS_Store
Thumbs.db
```

Part 4 — Commit, push, and open a pull request (10 minutes)

Step 10 — Inspect and commit the change

Use this tool	Where you should be	Visible finish line
GitHub Desktop	Changes tab on branch docs/week-1-setup	One local commit named docs: establish Week 1 architectural trail.

1. Return to GitHub Desktop. The left panel should list the files you changed or created.
2. Select each file and inspect the green added lines. Confirm there are no names of classmates, private data, passwords, tokens, or API keys.
3. At the lower left, in Summary, type: docs: establish Week 1 architectural trail
4. Leave Description blank or type: Adds repository purpose, charter, AI log, placeholders, and secret exclusions.
5. Select Commit to docs/week-1-setup.
6. At the top, select Push origin or Publish branch. This sends the commit to GitHub.

THREE DIFFERENT SAVES

Save writes the file in VS Code. Commit records a named local checkpoint. Push sends committed checkpoints to GitHub. Students often save but forget to commit or commit but forget to push.

Step 11 — Open a linked pull request

Use this tool	Where you should be	Visible finish line
GitHub website	Your repository on branch docs/week-1-setup	Open pull request with base main and compare docs/week-1-setup.

1. In GitHub Desktop, select Preview Pull Request or View on GitHub. Alternatively, open the repository website and select Compare & pull request in the yellow banner.
2. Confirm base: main and compare: docs/week-1-setup. If both menus say main, stop and correct the compare branch.
3. Title: Week 1: establish my architectural trail
4. In the description, type Closes #1 on its own line. Add: Creates the safe repository structure and working rules for the semester.
5. Select Create pull request.

6. On the pull request page, confirm Issue #1 appears in the Development area or is linked by Closes #1.
7. Official pull-request directions: <https://docs.github.com/en/pull-requests/how-tos/create-pull-requests/creating-a-pull-request>.

Part 5 — Reciprocal peer review (10 minutes)

EQUAL PRACTICE, NOT UNEQUAL ROLES

The instructor pairs students A and B. Student A reviews B; Student B reviews A. Every student is therefore both an author and a reviewer. No student receives a different team-lead assignment.

Step 12 — Give your assigned peer temporary repository access

Use this tool	Where you should be	Visible finish line
GitHub website	Your repository > Settings	Your assigned peer appears as a collaborator; you accept the reciprocal invitation.

1. Exchange GitHub usernames with the one peer assigned by the instructor. Do not exchange passwords.
2. In your repository, select Settings. If the window is narrow, the tab may appear under an overflow menu.
3. In the left sidebar, select Collaborators or Collaborators and teams.
4. Select Add people, search for the peer's exact GitHub username, and send the invitation.
5. Open the invitation to the peer's repository and accept it. Each student performs both the invite and accept actions.

Step 13 — Review one peer pull request

Use this tool	Where you should be	Visible finish line
GitHub website	Assigned peer's repository > Pull requests	You submitted one useful Comment review; your own pull request received one.

1. Open the assigned peer's repository and select Pull requests.
2. Open the Week 1 pull request and read the description and linked issue.
3. Select Files changed.

4. Check four things: file paths are correct; README explains the repository; privacy rules are explicit; no obvious secret or private information appears.
5. Hover beside a useful line and select the blue comment icon. Leave one specific comment.
Example: The boundary is clear; please add that unapproved IT documents are also out of scope.
6. Select Start a review or Add review comment.
7. Select Review changes, choose Comment, type a one-sentence summary, then select Submit review. Use Comment rather than Approve if GitHub does not allow approval because of repository permissions.
8. Official review directions: <https://docs.github.com/en/pull-requests/how-tos/review-pull-requests/reviewing-proposed-changes-in-a-pull-request>.

Step 14 — Respond, revise if needed, and merge

Use this tool	Where you should be	Visible finish line
GitHub website, VS Code, and GitHub Desktop	Your own pull request	Peer comment answered, any correction pushed, pull request merged, Issue #1 closed.

1. Open your own pull request and read the peer comment.
2. Reply with either: Updated as suggested, or I kept the current wording because <brief reason>.
3. If a change is needed, edit the file in VS Code, save, commit in GitHub Desktop with a short message, and Push origin. The same pull request updates automatically.
4. When the review is resolved, select Merge pull request, then Confirm merge.
5. Return to Issues. Because the pull request used Closes #1, Issue #1 should now be closed. If it remains open, add a comment linking the merged pull request and close it manually.
6. In GitHub Desktop, switch Current Branch to main and select Fetch origin, then Pull origin. Your local main now receives the merged work.

THE ARCHITECTURAL TRAIL NOW EXISTS

Issue explains why → branch isolates the work → files contain the artifact → commit records a checkpoint → pull request exposes the change → peer review adds a human check → merge updates main.

Part 6 — Create the semester Project board (8 minutes)

Step 15 — Create four identical starter issues

Use this tool	Where you should be	Visible finish line
GitHub website	Your repository > Issues	Issues #2–#4 exist in addition to the completed Week 1 issue.

Create three new blank issues. You already completed Issue #1. For each issue, select New issue, paste the title and one-sentence finish line below, assign yourself, and submit.

Issue title	One-sentence finish line
Week 2: capture approved discovery evidence	Add only anonymized, instructor-approved team evidence and label assumptions.
Week 3: create Requirements Specification v1 and AI workspaces	Create the requirements file, configure ChatGPT and Claude, record one human decision.
Week 4: compare CampusConnect RAG architectures	Compare local LM Studio and hosted prototype paths using a small risk spike.

Step 16 — Create and link the board

Use this tool	Where you should be	Visible finish line
GitHub website	Your profile Projects area, then your repository Projects tab	CampusConnect Semester Board is linked and displays Week 1–4 issues.

1. Open your GitHub profile menu, select Your projects, then select New project. If the label differs, use the direct Projects area from your profile.
2. Under Start from scratch, select Board.
3. Project name: CampusConnect Semester Board
4. Select Create project.
5. At the bottom of a column, select Add item or the plus sign. Search for your repository and add the four Week 1–4 issues.
6. Move the Week 1 issue to Done. Leave the Week 2–4 issues in Todo or Backlog.
7. Return to the repository. Select Projects > Link a project, search CampusConnect Semester Board, and select it.
8. Official create-project directions: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/creating-projects/creating-a-project>.

9. Official add-items directions: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/managing-items-in-your-project/adding-items-to-your-project>.
10. Official link-project directions: <https://docs.github.com/en/issues/planning-and-tracking-with-projects/managing-your-project/adding-your-project-to-a-repository>.

KEEP THE BOARD SIMPLE

Do not create four views, custom formulas, roadmaps, automation rules, or a complex label taxonomy today. Todo, In Progress, and Done are enough for Week 1.

Part 7 — Verify and submit (2 minutes)

Required evidence	Check
Individual private repository URL	Complete / Missing
README, .gitignore, docs, src, and tests visible on main	Complete / Missing
Closed Issue #1 and merged Week 1 pull request	Complete / Missing
One peer review you gave and one review you received	Complete / Missing
Linked CampusConnect Semester Board with Week 1–4 issues	Complete / Missing
One redacted screenshot of the final repository file list	Complete / Missing

Common problems and plain-language fixes

What you see	What it usually means	What to do
GitHub Desktop says No local changes	VS Code may have opened the wrong folder or the file was not saved.	In VS Code, use File > Open Folder and choose the cloned CampusConnect folder; save again.
Current Branch says main	You are about to change the stable branch.	Stop. In GitHub Desktop, select Current Branch > docs/week-1-setup.
GitHub does not show the new commit	The commit is still local.	In GitHub Desktop, select Push origin or Publish branch.
The PR compares main to main	The compare branch is wrong.	Set base = main and compare = docs/week-1-setup.
Peer cannot open the private repository	The collaborator invitation is missing or unaccepted.	Recheck Settings > Collaborators and ask the peer to accept the GitHub notification.
A folder disappeared	Git does not preserve an empty folder.	Create a small README.md inside the folder.

Browser-only fallback

If a managed computer blocks GitHub Desktop or VS Code, complete the same issue, branch, file, commit, pull-request, review, and board steps on github.com. On the branch, use Add file > Create new

file. In the filename box, typing docs/PROJECT_CHARTER.md creates both the docs folder and the file. Follow the official file directions below and record Browser-only fallback in Issue #1.

Official browser file directions: <https://docs.github.com/en/repositories/working-with-files/managing-files/creating-new-files>.

Mini-rubric — 20 points

Criterion	Pts	Full-credit evidence
Repository and local setup	4	Correct individual private repository; clone visible in Desktop/VS Code or documented browser fallback.
Files and safety boundary	4	README, charter, AI log, placeholders, and secret exclusions are complete and appropriate.
GitHub flow	5	Issue → linked branch → coherent commit → pull request → merge is visible.
Reciprocal review	3	Student gives and receives one specific review and responds professionally.
Project board and submission	4	Week 1–4 issues are visible on the linked board and all required links/evidence are submitted.

Official reference: GitHub Flow

The semester workflow practiced here is documented at <https://docs.github.com/en/get-started/using-github/github-flow>.